EXAMEN PARCIAL 02

Tema: PARTE II

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza (Teoría grupo B) ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Arq. de computadoras Semestre: IV Código: 1702227

Laboratorio	Tema	Duración
02	PARTE II	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	08/11/2026	08/11/2026

Índice

1. **Pregunta 1: Decimal a IEEE 754 de precisión simple** **3**
2. **Pregunta 2: IEEE 754 de precisión simple a decimal** **3**
3. **Pregunta 3: Memoria jerárquica** **3**
4. **Pregunta 4: Ejecución de la instrucción `SW #2AB`** **5**
5. **Pregunta 5: Franjas horizontales de colores en assembler Intel** **6**
6. **Pregunta 6: Manipulación de bits del registro CX** **8**

1. Pregunta 1: Decimal a IEEE 754 de precisión simple

Se representa $13{,}15625_{10}$ en el formato IEEE 754 de precisión simple (32 bits: 1 bit de signo, 8 bits de exponente con sesgo 127 y 23 bits de mantisa).

- **Signo:** el número es positivo, por lo tanto $S = 0$.
- **Parte entera:** $13_{10} = 1101_2$.
- **Parte fraccionaria:** se multiplica por 2 de forma sucesiva y se toma la parte entera de cada resultado.

Operación	Resultado	Bit
$0{,}15625 \times 2$	0,3125	0
$0{,}3125 \times 2$	0,625	0
$0{,}625 \times 2$	1,25	1
$0{,}25 \times 2$	0,5	0
$0{,}5 \times 2$	1,0	1

- Entonces $0{,}15625_{10} = 0{,}00101_2$ y $13{,}15625_{10} = 1101{,}00101_2$.
- **Normalización:** $1101{,}00101_2 = 1{,}10100101_2 \times 2^3$ (la coma se desplaza 3 lugares a la izquierda), es decir, $e = 3$.
- **Exponente sesgado:** $E = e + 127 = 3 + 127 = 130_{10} = 10000010_2$.
- **Mantisa:** son los bits que siguen al 1 implícito, 10100101_2, completados con ceros hasta los 23 bits.

Signo (1 bit)	Exponente (8 bits)	Mantisa (23 bits)
0	10000010	10100101000000000000000

Respuesta: $13{,}15625_{10} = 0\ 10000010\ 10100101000000000000000_2 = 41528000_{16}$.
Comprobación: $1{,}10100101_2 = 1 + \frac{1}{2} + \frac{1}{8} + \frac{1}{64} + \frac{1}{256} = 1{,}64453125$ y $1{,}64453125 \times 2^3 = 13{,}15625$.

2. Pregunta 2: IEEE 754 de precisión simple a decimal

Número dado: 1 01100110 10100000000000000000000 (signo, exponente y mantisa).

- **Signo:** $S = 1$, por lo tanto el número es negativo.
- **Exponente:** $E = 01100110_2 = 64 + 32 + 4 + 2 = 102_{10}$, entonces $e = E - 127 = 102 - 127 = -25$.
- **Mantisa:** $M = 101\,000\ldots0_2$, por lo que el significando (con el 1 implícito) es $1{,}101_2 = 1 + \frac{1}{2} + \frac{1}{8} = 1{,}625$.

$$N = (-1)^S \times 1.M \times 2^e = -1{,}625 \times 2^{-25} = -\frac{13}{2^{28}} = -\frac{13}{268\,435\,456}$$

Respuesta: $N \approx -4{,}8429 \times 10^{-8}$ (en hexadecimal, $B3500000_{16}$).

3. Pregunta 3: Memoria jerárquica

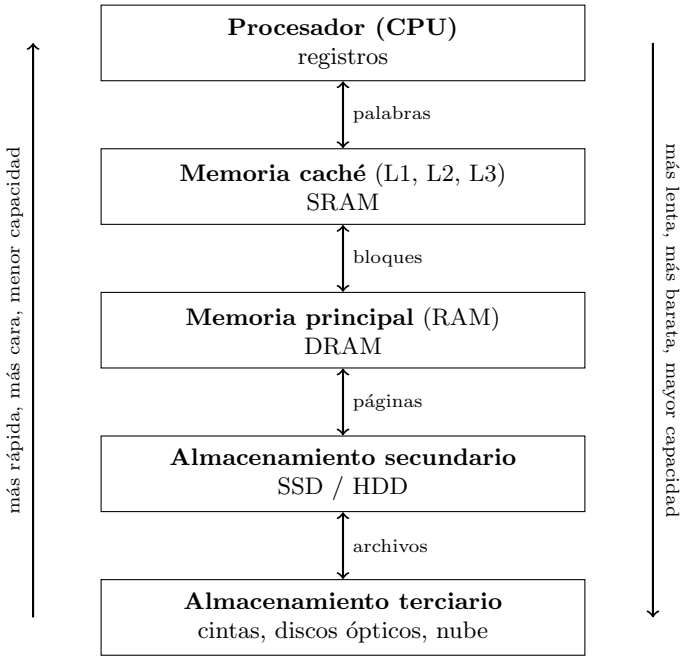
La *memoria jerárquica* organiza la memoria del computador en varios niveles construidos con tecnologías distintas. Los niveles cercanos al procesador son pequeños, muy rápidos y caros por bit; los niveles lejanos son grandes, lentos y baratos. Como ninguna tecnología es a la vez rápida, barata y de gran capacidad, se combinan para que el sistema se comporte como una memoria grande y casi tan rápida como la más veloz.

La jerarquía funciona gracias al *principio de localidad*:

- **Localidad temporal:** un dato usado recientemente tiende a volver a usarse pronto.

- **Localidad espacial:** después de acceder a una dirección es probable acceder a las cercanas, por eso se copian bloques completos y no palabras sueltas.

Cuando el procesador necesita un dato, lo busca primero en el nivel más cercano. Si está (acierto) lo usa; si no está (fallo), se trae un bloque desde el nivel siguiente. Cada nivel contiene un subconjunto de los datos del nivel inferior.



Esquema de un procesador conectado a una memoria jerárquica.

Niveles que se encuentran:

- **Registros de la CPU:** están dentro del procesador y guardan operandos y resultados inmediatos. Son el almacenamiento más rápido, pero muy pocos.
- **Memoria caché (L1, L2, L3):** memoria SRAM entre la CPU y la RAM que guarda copias de los bloques más usados. La administra el hardware; L1 es la más pequeña y rápida, y L3 la más grande y lenta.
- **Memoria principal (RAM):** memoria DRAM que contiene los programas y datos en ejecución. El sistema operativo la administra con memoria virtual (paginación y MMU).
- **Secundario (SSD o HDD):** memoria no volátil que guarda archivos y programas y recibe las páginas que no caben en la RAM.
- **Almacenamiento terciario:** cintas, discos ópticos o la nube, para copias de seguridad y datos que casi no se usan.

Nivel	Tecnología	Acceso	Capacidad	Gestión
Registros	Flip-flops	< 1 ns	cientos de bytes	Compilador
Caché (L1–L3)	SRAM	1 a 30 ns	KB a MB	Hardware
Memoria principal	DRAM	50 a 100 ns	GB	S.O.
Secundario	SSD / HDD	0.1 a 10 ms	GB a TB	S.O.
Terciario	Cinta / nube	s a min	TB a PB	Operador

Valores típicos y orientativos (S.O. = sistema operativo).

Al bajar en la jerarquía aumentan la capacidad y el tiempo de acceso, y disminuye el costo por bit.

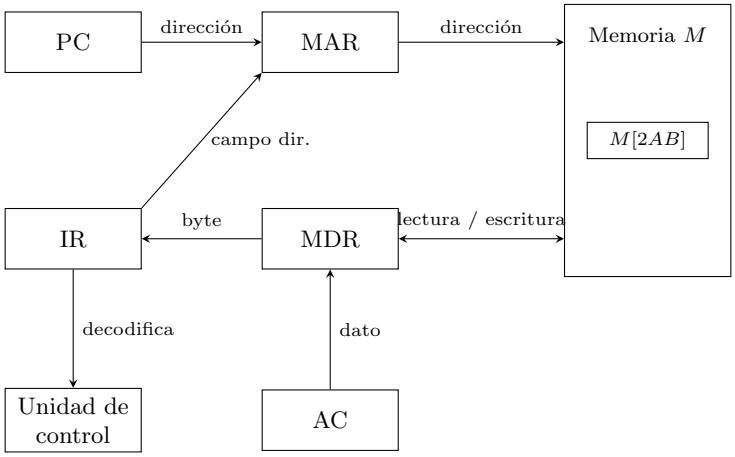
4. Pregunta 4: Ejecución de la instrucción `SW #2AB`

La instrucción `SW #2AB` (*store word*) guarda el contenido del acumulador en la posición de memoria $2AB_{16}$ (el símbolo # indica que $2AB$ es una dirección de memoria). En notación de transferencia de registros:

$$M[2AB] \leftarrow AC$$

Según la secuencia del computador elemental explicada en clase, la instrucción se ejecuta en cuatro fases: búsqueda (*fetch*), decodificación y lectura de operandos, ejecución de la operación y almacenamiento del resultado. Como la instrucción tiene 16 bits y la memoria transfiere 8 bits por acceso, la búsqueda requiere dos lecturas consecutivas. Los elementos que intervienen son:

- **PC (contador de programa):** contiene la dirección de la siguiente instrucción.
- **MAR (registro de dirección de memoria):** coloca la dirección en el bus de direcciones.
- **MDR (registro de datos de memoria):** recibe o entrega el dato que viaja por el bus de datos.
- **IR (registro de instrucción):** guarda la instrucción que se está ejecutando.
- **AC (acumulador):** contiene el dato que se va a almacenar.
- **Unidad de control (UC):** decodifica la instrucción y genera las señales de control.



Registros y transferencias que intervienen en el ciclo de instrucción.

Paso	Micro-operación	Qué ocurre
1. Búsqueda de la instrucción (*fetch*)		
1	$MAR \leftarrow PC$	Se coloca en MAR la dirección del primer byte de la instrucción.
2	$MDR \leftarrow M[MAR]$	La memoria entrega el primer byte (8 bits) a MDR.
3	$IR[\text{byte } 1] \leftarrow MDR$	El primer byte se guarda en la parte correspondiente de IR.
4	$PC \leftarrow PC + 1$	PC apunta al segundo byte de la instrucción.
5	$MAR \leftarrow PC$	Se coloca en MAR la dirección del segundo byte.
6	$MDR \leftarrow M[MAR]$	La memoria entrega el segundo byte (8 bits) a MDR.
7	$IR[\text{byte } 2] \leftarrow MDR$	Se completa la instrucción de 16 bits en IR.
8	$PC \leftarrow PC + 1$	PC apunta a la instrucción siguiente.
2. Decodificación y lectura de operandos		
–	Decodificar IR	La UC reconoce `SW` y obtiene la dirección $2AB$ del campo de la instrucción. El dato fuente ya está en AC.
3. Ejecución de la operación		
9	$MAR \leftarrow IR[\text{dir}]$	La dirección de destino ($2AB$) se coloca en MAR.
10	$MDR \leftarrow AC$	El dato (o el byte que corresponda) se coloca en MDR para transferirlo a memoria.
4. Almacenamiento del resultado		
11	$M[MAR] \leftarrow MDR$	Se activa la escritura y se almacena el dato en la dirección indicada. Si la palabra ocupa más de 8 bits, se repite la transferencia para sus bytes restantes.

La búsqueda realiza dos lecturas de memoria porque cada acceso entrega 8 bits y la instrucción ocupa 16 bits; por ello PC se incrementa dos veces. La dirección $2AB$ se obtiene al decodificar el campo de dirección y el dato fuente se toma de AC. En la fase final se escribe en memoria; si el tamaño del dato almacenado supera los 8 bits, se necesitan transferencias adicionales para los bytes restantes, de acuerdo con el formato y el orden de bytes definidos para el computador. El acumulador no se modifica.

Nota: Se mantienen los nombres MAR, MDR, IR, PC y AC del esquema. La división de la instrucción en sus dos bytes y el posible número de escrituras de datos dependen del formato y del tamaño de palabra usados en el computador explicado en clase.

5. Pregunta 5: Franjas horizontales de colores en assembler Intel

Idea. La pantalla de texto de 80×25 caracteres dispone de 16 colores (numerados de 0 a 15). La imagen del enunciado muestra 16 franjas, una por color: cada franja ocupa una fila y 16 columnas, y la primera es negra (por eso no se distingue del fondo). Se usan dos funciones de la interrupción `INT 10h` del BIOS:

- `AH = 02h`: posiciona el cursor (`DH` = fila, `DL` = columna, `BH` = página).
- `AH = 09h`: escribe el carácter de `AL` con el atributo de `BL`, repetido `CX` veces desde la posición del cursor.

Cada franja se dibuja con el carácter de bloque lleno (`0DBh`) cuyo color de primer plano es el número de la fila. De este modo, la fila n se pinta con el color n y no hace falta una tabla de colores.

Fila	Color	Nombre	Fila	Color	Nombre
0	0h	Negro	8	8h	Gris oscuro
1	1h	Azul	9	9h	Azul claro
2	2h	Verde	10	Ah	Verde claro
3	3h	Cian	11	Bh	Cian claro
4	4h	Rojo	12	Ch	Rojo claro
5	5h	Magenta	13	Dh	Magenta claro
6	6h	Marrón	14	Eh	Amarillo
7	7h	Gris claro	15	Fh	Blanco

Programa (`franjas.asm`):

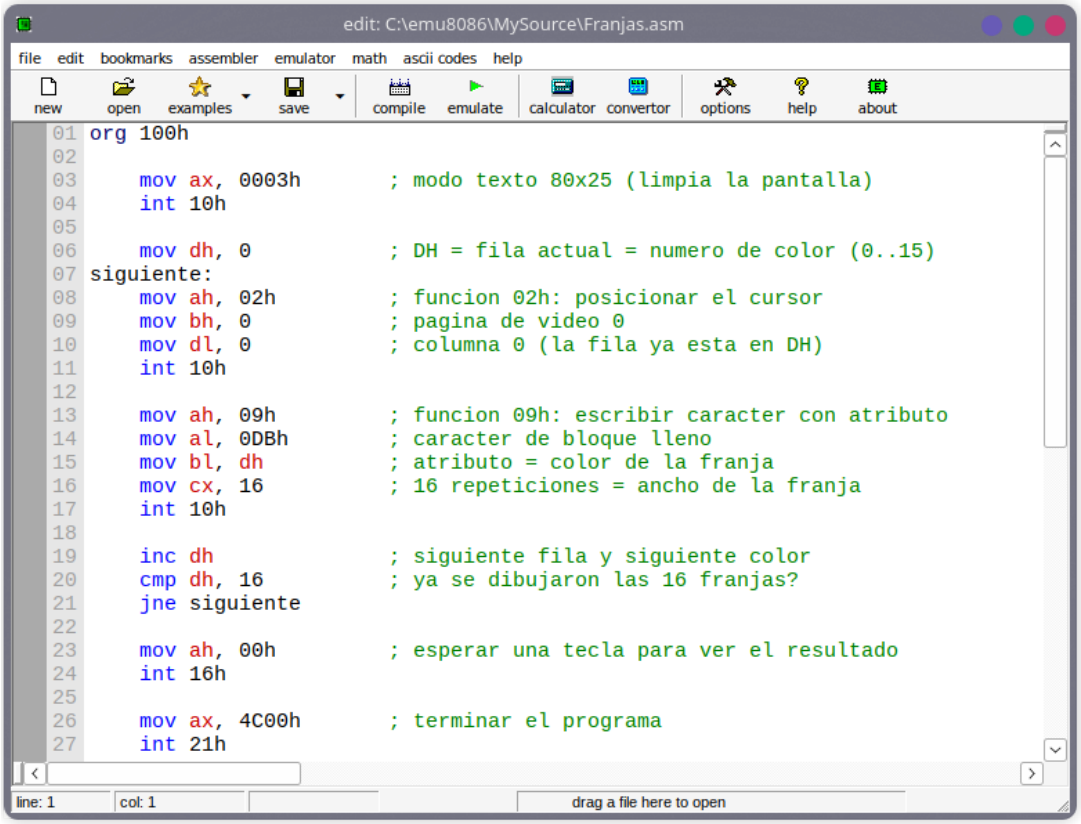


Figura 1: Franjas.asm en EMU8086

Funcionamiento.

- Se fija el modo de texto 80×25 con `AX = 0003h`, lo que también limpia la pantalla.
- `DH` lleva la fila y, a la vez, el número de color (0 a 15). En cada vuelta se posiciona el cursor en la columna 0 de esa fila.
- Se escriben 16 bloques llenos con el atributo `BL = DH`, es decir, una franja de 16 columnas del color de esa fila.
- Se incrementa `DH` y se repite hasta completar 16 franjas (filas 0 a 15).
- Se espera una tecla (`INT 16h`) para poder ver la pantalla y se termina con `INT 21h, AH = 4Ch`.

Se usa el color de primer plano porque funciona igual en cualquier entorno; los colores de fondo 8 a 15 dependen de si el modo de parpadeo del texto está activado.

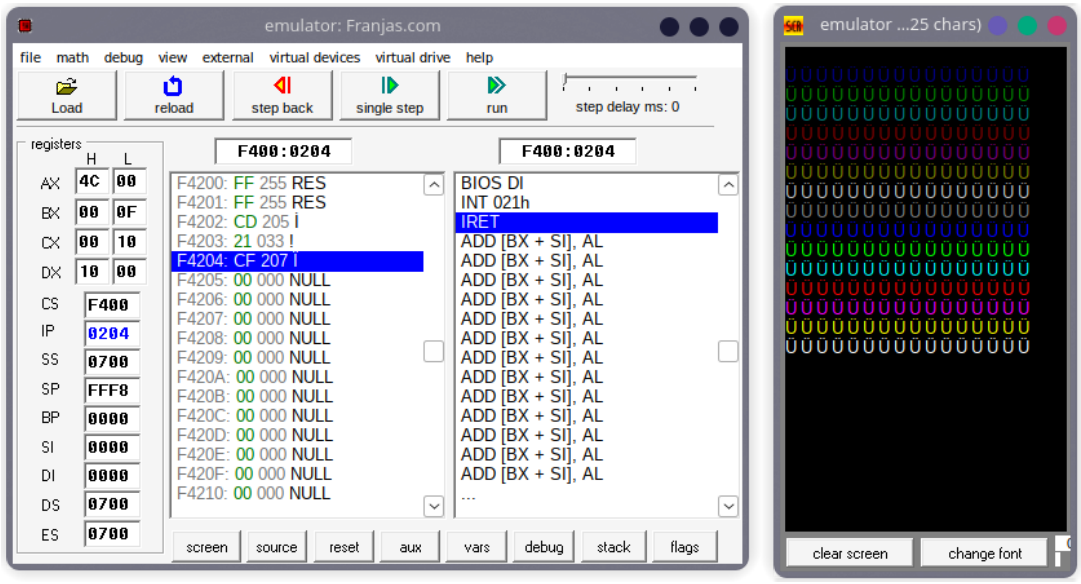


Figura 2: Ejecución de Franjas.asm en EMU8086

6. Pregunta 6: Manipulación de bits del registro CX

Se pide, sobre el dato de CX (bits D7 a D0): invertir los bits impares D5, D3 y D1; borrar los bits pares D6, D4 y D2; y poner en 1 los bits D7 y D0. Cada acción se resuelve con una instrucción lógica y una máscara:

- **Invertir** bits: `XOR` con 1 en los bits a invertir. Máscara $0010\ 1010_2 = 2Ah$ (D5, D3, D1).
- **Borrar** bits: `AND` con 0 en los bits a borrar y 1 en el resto. Máscara $1010\ 1011_2 = ABh$ (D6, D4, D2 en cero). Como CX tiene 16 bits, se usa $FFABh$ para no alterar el byte alto.
- **Poner en 1** bits: `OR` con 1 en los bits a poner. Máscara $1000\ 0001_2 = 81h$ (D7 y D0).

Bit	D7	D6	D5	D4	D3	D2	D1	D0
Acción	poner 1	borrar	invertir	borrar	invertir	borrar	invertir	poner 1

Las tres operaciones afectan a bits distintos, por lo que el orden en que se aplican no cambia el resultado.

Programa (`bits.asm`), con el dato de ejemplo del enunciado ($CX = 00B1h$, es decir, $1011\ 0001_2$ en D7 a D0):

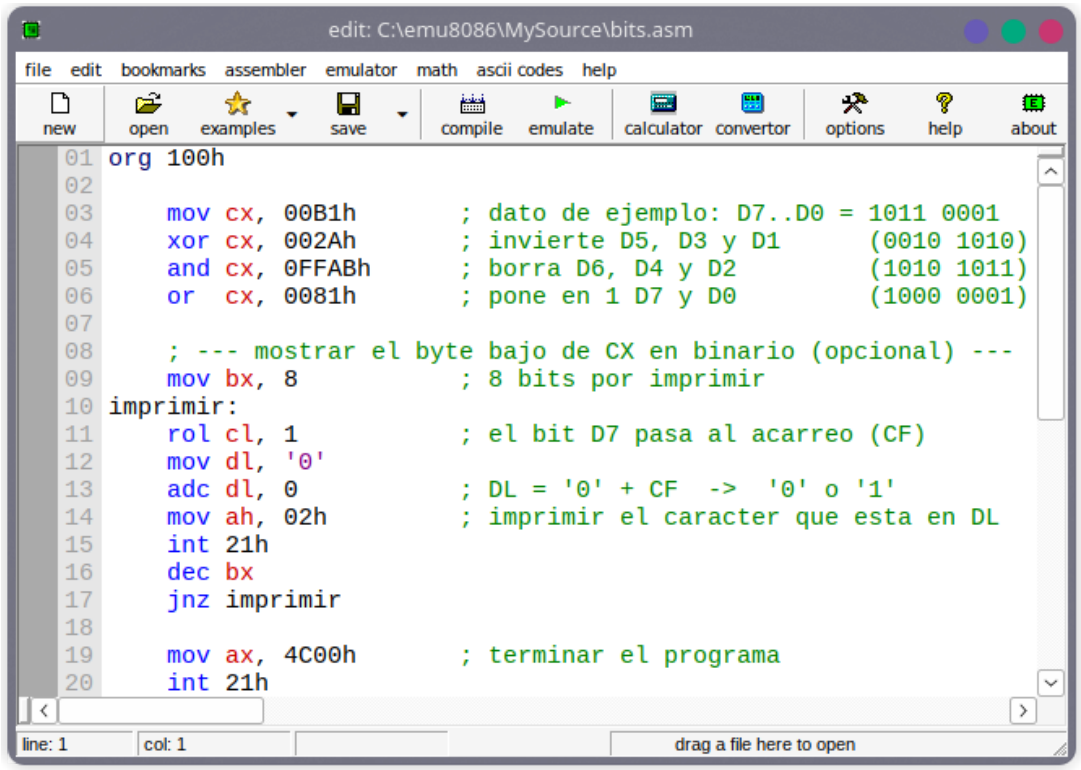


Figura 3: bits.asm en EMU8086

Las cuatro primeras instrucciones son la solución; el bloque `imprimir` solo sirve para ver el resultado en DOSBox, donde no hay ventana de registros.

Comprobación con el ejemplo:

Paso	**Efecto**	**Byte bajo de CX**
Dato inicial	—	$1011\ 0001_2 = B1h$
`XOR CX, 002Ah`	invierte D5, D3 y D1	$1001\ 1011_2 = 9Bh$
`AND CX, 0FFABh`	borra D6, D4 y D2	$1000\ 1011_2 = 8Bh$
`OR CX, 0081h`	pone en 1 D7 y D0	$1000\ 1011_2 = 8Bh$

En este ejemplo el `OR` no cambia el valor porque D7 y D0 ya estaban en 1.

Respuesta: el resultado es $CX = 008Bh$, es decir, $1000\ 1011_2$ en D7 a D0. Para cualquier otro dato el byte alto no cambia; por ejemplo, $CX = 1234h$ queda en $128Bh$.

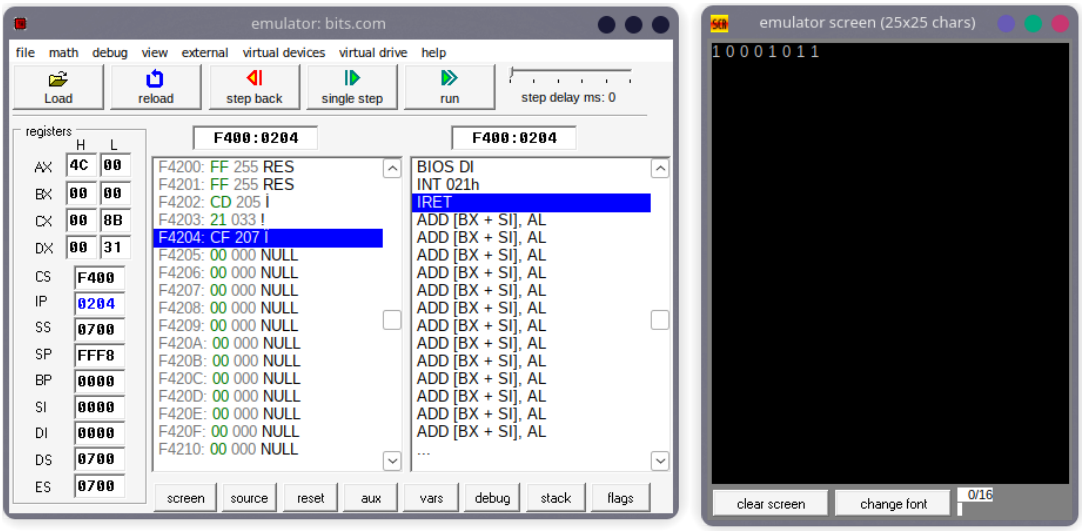


Figura 4: Ejecución de bits.asm en EMU8086